

Question_10:

Code:

```
1 // There are n cities and m undirected streets. Find the minimum number
2 // of streets that must be closed so that there is no route from city 1 to n.
3 #include <iostream>
4 #include <cstring>
5 using namespace std;
6
7 #define MAXN 505
8
9 int capacity[MAXN][MAXN];
10 int adj[MAXN][MAXN];
11 int deg[MAXN];
12 int parent_node[MAXN];
13 int visited[MAXN];
14
15 int bfs(int n, int s, int t)
16 {
17     memset(visited, 0, sizeof(visited));
18     int queue[MAXN];
19     int head = 0, tail = 0;
20
21     queue[tail++] = s;
22     visited[s] = 1;
23     parent_node[s] = -1;
24     while (head < tail)
25     {
26         int u = queue[head++];
27         if (u == t)
28             return 1;
29         for (int i = 0; i < deg[u]; i++)
30         {
31             int v = adj[u][i];
32             if (!visited[v] && capacity[u][v] > 0)
33             {
34                 visited[v] = 1;
35                 parent_node[v] = u;
36                 queue[tail++] = v;
37             }
38         }
```

```
39     }
40     return 0;
41 }
42 void dfs_reach(int u)
43 {
44     visited[u] = 1;
45     for (int i = 0; i < deg[u]; i++)
46     {
47         int v = adj[u][i];
48         if (!visited[v] && capacity[u][v] > 0)
49             dfs_reach(v);
50     }
51 }
52 void link(int u, int v)
53 {
54     capacity[u][v] = 1;
55     capacity[v][u] = 1;
56     adj[u][deg[u]++] = v;
57     adj[v][deg[v]++] = u;
58 }
59 int main()
60 {
61     int n, m;
62     if (cin >> n >> m)
63     {
64         int edges[1005][2];
65         for (int i = 0; i < m; i++)
66         {
67             cin >> edges[i][0] >> edges[i][1];
68             link(edges[i][0], edges[i][1]);
69         }
70         while (bfs(n, 1, n))
71         {
72             int curr = n;
73             while (curr != 1)
```

```
74         {
75             int prev = parent_node[curr];
76             capacity[prev][curr] -= 1;
77             capacity[curr][prev] += 1;
78             curr = prev;
79         }
80     }
81     memset(visited, 0, sizeof(visited));
82     dfs_reach(1);
83     int cut_count = 0;
84     int cut_edges[1005][2];
85     for (int i = 0; i < m; i++)
86     {
87         int u = edges[i][0];
88         int v = edges[i][1];
89         if ((visited[u] && !visited[v]) ||
90             (visited[v] && !visited[u]))
91         {
92             cut_edges[cut_count][0] = u;
93             cut_edges[cut_count][1] = v;
94             cut_count++;
95         }
96     }
97     cout << cut_count << "\n";
98     for (int i = 0; i < cut_count; i++)
99         cout << cut_edges[i][0] << " " << cut_edges[i][1] << "\n";
100 }
101 return 0;
102 }
```

Output:

Test Case_1:

```
4 5
1 2
1 3
2 3
3 4
1 4
2
3 4
1 4
PS C:\Users\Prabin Yadav\
```

Test Case_2:

```
4 5
1 1
2 2
2 3
3 4
1 4
1
1 4
PS C:\Users\Prabin Yadav\
```